

Estructura de datos

Listas como arreglos

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2026-II

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

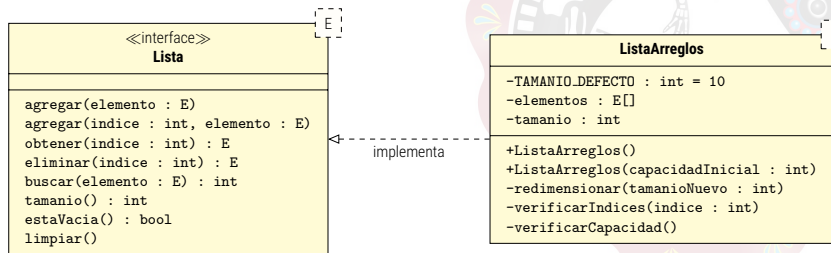
- 1 Implementación de listas usando arreglos
- 2 Agregar elemento
- 3 Obtener elemento
- 4 Eliminar elemento
- 5 Buscar elemento
- 6 Tamaño de la lista
- 7 Lista vacía
- 8 Limpiar lista



Diagrama de clases

Objetivo

Implementar el TDA **Lista** utilizando un **arreglo** como estructura interna de almacenamiento.



Implementación de ListaArreglos<E>

Clase ListaArreglos

Implementación de la interfaz Lista utilizando genéricos

```
public class ListaArreglos<E> implements Lista<E> {  
    private static final int TAMANIO_DEFECTO = 10;  
  
    private E[] elementos;  
    private int tamaño;  
  
    // ...  
}
```

Atributos

- **elementos**: arreglo que **almacena** los elementos.
- **tamaño**: número de **elementos** actualmente **almacenados**.
- **TAMANIO_DEFECTO**: **capacidad inicial** de la lista.

Creación de una ListaArreglos

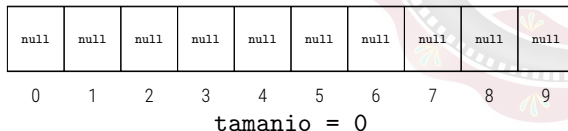
La implementación proporciona **dos formas** de crear una lista.

Capacidad por defecto

- 1: crear arreglo de tamaño 10
- 2: `tamano ← 0`

```
new ListaArreglos<>()
```

```
elementos.length = 10
```



Creación de una ListaArreglos definida

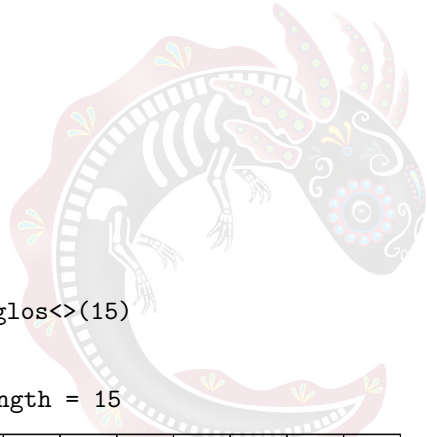
Capacidad especificada

Require: capacidadInicial

- 1: **if** *capacidadInicial* < 0 **then**
- 2: lanzar excepción
- 3: **end if**
- 4: crear arreglo
- 5: **tamaño** $\leftarrow$ 0

```
new ListaArreglos<>(15)
```

```
elementos.length = 15
```

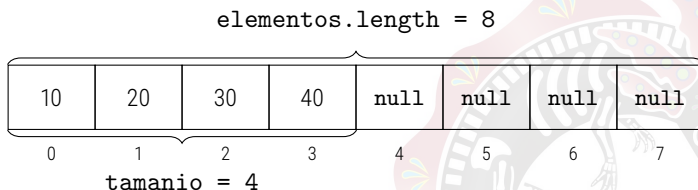


null	null	null	null	null	null	null	null	null	null	null	null	null	null	null
------	------	------	------	------	------	------	------	------	------	------	------	------	------	------

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14

tamaño = 0

Capacidad y tamaño lógico



Dos conceptos diferentes

Capacidad: número de **posiciones disponibles** en el arreglo.

Tamaño: número de **elementos** que **pertenecen actualmente** a la lista.

$$0 \leq \text{tamaño} \leq \text{elementos.length}$$

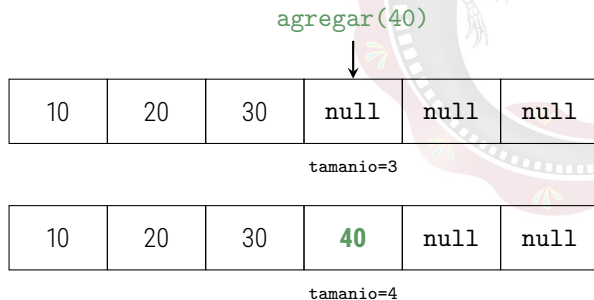
Método agregar(elemento)

Definición

Inserta un nuevo elemento **al final** de la lista.

Require: elemento

- 1: verificar capacidad
- 2: `elementos[tamano] ← elemento`
- 3: `tamano ← tamano + 1`



Verificar la capacidad

Antes de insertar un elemento:

- 1: **if** tamaño = elementos.length **then**
- 2: capacidadNueva $\leftarrow$ elementos.length $\times$ 2
- 3: redimensionar(capacidadNueva)
- 4: **end if**

Capacidad = 4

10	20	30	40
----	----	----	----

$\xrightarrow{\times 2}$

Capacidad = 8

10	20	30	40				
----	----	----	----	--	--	--	--

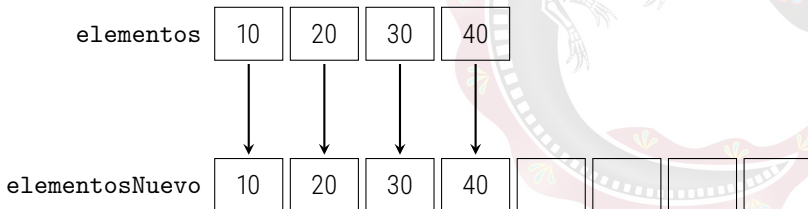
¡Importante!

El arreglo **solamente** se **redimensiona** cuando está **completamente lleno**.

Método `redimensionar()`

Require: `tamanoNuevo`

- 1: crear `elementosNuevo[tamanoNuevo]`
- 2: **for** $i \leftarrow 0$ **hasta** $\text{tamano} - 1$ **do**
- 3: `elementosNuevo[i]  $\leftarrow$  elementos[i]`
- 4: **end for**
- 5: `elementos  $\leftarrow$  elementosNuevo`



`elementos = elementosNuevo`

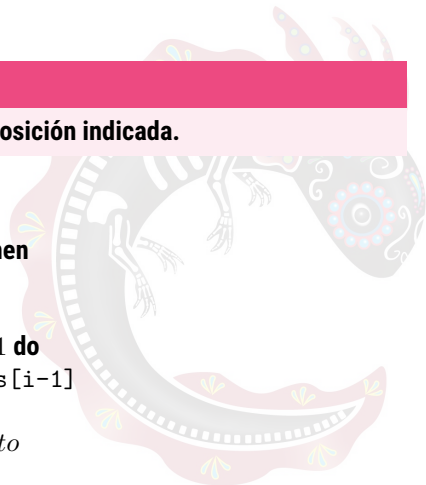
Método agregar(indice, elemento)

Definición

Inserta en la lista un nuevo elemento en la **posición indicada**.

Require: indice, elemento

```
1: verificar capacidad
2: if indice < 0 or indice > tamano then
3:   agregar(elemento)
4: else
5:   for i ← tamano hasta indice + 1 do
6:     elementos[i] ← elementos[i-1]
7:   end for
8:   elementos[indice] ← elemento
9:   tamano ← tamano + 1
10: end if
```



Método agregar(indice, elemento)

¡Importante!

Si el **índice** está **fuera del intervalo**

$$0 \leq \text{índice} \leq \text{tamaño}$$

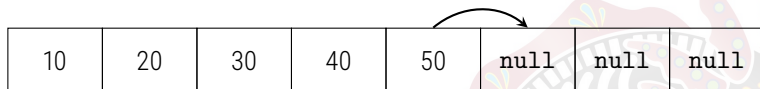
el elemento se **agrega al final de la lista.**

Ejemplo

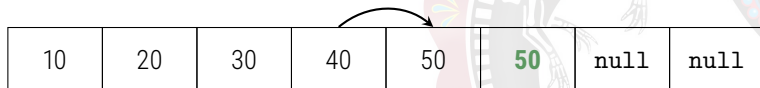
Ejecutar el método

agregar(2,25)

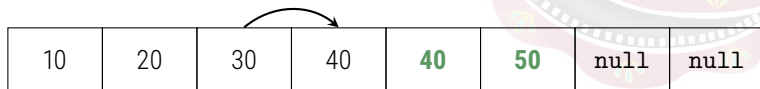
Desplazamiento para insertar



indice=2

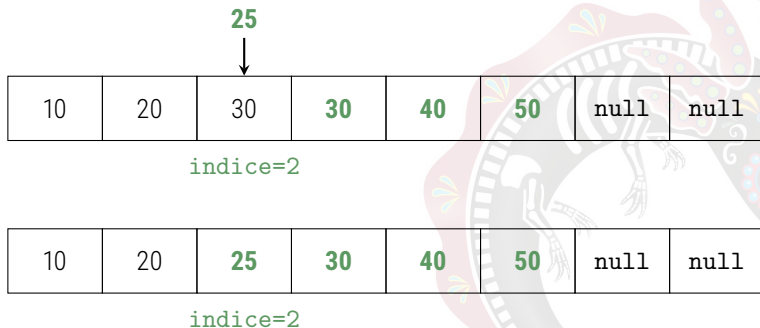


indice=2



indice=2

Desplazamiento para insertar



- 1 Desplazar 50 hacia la derecha.
- 2 Desplazar 40 hacia la derecha.
- 3 Desplazar 30 hacia la derecha.
- 4 Almacenar 25 en la posición 2.

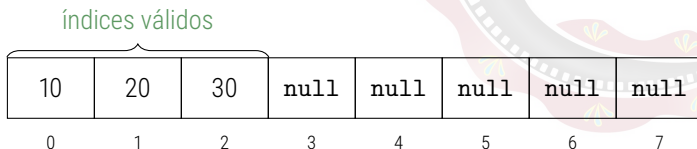
Verificación de índices

Los métodos `obtener()` y `eliminar()` solo pueden trabajar con posiciones que contienen elementos.

$$0 \leq \text{indice} < \text{tamaño}$$

Require: indice

- 1: **if** $\text{indice} < 0$ **or** $\text{indice} \geq \text{tamaño}$ **then**
- 2: lanzar `IndexOutOfBoundsException`
- 3: **end if**



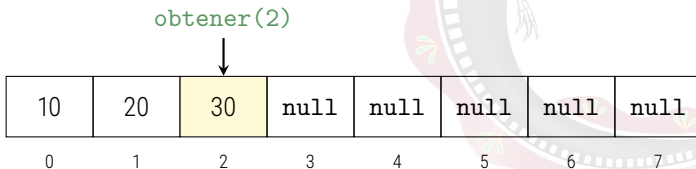
Método obtener(indice)

Definición

Devuelve el **elemento de la lista** indicado por su posición.

Require: indice

- 1: verificarIndice(indice)
- 2: **return** elementos[indice]



`obtener(2) → 30`

Método `eliminar(indice)`

Definición

Elimina un elemento de la lista indicado por su posición.

Require: índice

```
1: verificarIndice(indice)
2: eliminado  $\leftarrow$  elementos[indice]
3: for  $i \leftarrow indice$  hasta tamaño - 2 do
4:   elementos[i]  $\leftarrow$  elementos[i+1]
5: end for
6: tamaño  $\leftarrow$  tamaño - 1
7: return eliminado
```

¿Por qué guardar `eliminado`?

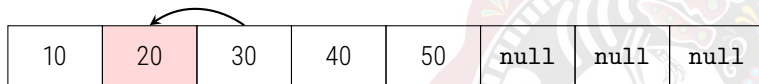
El método debe regresar una referencia al elemento que pertenecía a la posición eliminada.



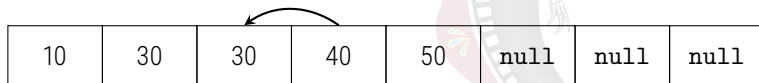
Desplazamiento al eliminar

Ejecutamos:

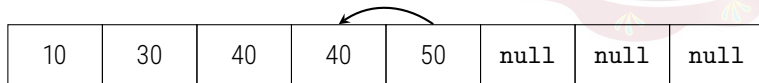
`eliminar(1)`



`indice=1`



`indice=1`



`indice=1`

Desplazamiento al eliminar

10	30	40	50	50	null	null	null
----	----	----	----	----	------	------	------

indice=1

pero:

tamaño = 4

Por tanto, la lista contiene lógicamente:

[10, 30, 40, 50]

Físicamente:

10	30	40	50	50	null	null	null
----	----	----	----	----	------	------	------

Método `buscar(elemento)`

Definición

Busca un elemento en la lista y devuelve el índice.

Require: elemento

```
1: for  $i \leftarrow 0$  hasta tamano - 1 do  
2:   if elementos[i].equals(elemento) then  
3:     return  $i$   
4:   end if  
5: end for  
6: return  $-1$ 
```

¡Importante!

Se utiliza `equals()` porque la lista almacena **objetos de tipo E**.

Método buscar(elemento)

buscar(30)

elemento=30

10	20	30	40	50	null	null	null
----	----	----	----	----	------	------	------

↑
i = 0

elemento=30

10	20	30	40	50	null	null	null
----	----	----	----	----	------	------	------

↑
i = 1

elemento=30

10	20	30	40	50	null	null	null
----	----	----	----	----	------	------	------

↑
i = 2

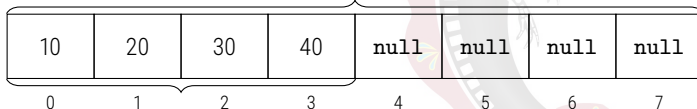
Método `tamaño()`

Definición

Devuelve el **tamaño lógico** de la lista.

1: **return** tamaño

`elementos.length = 8`



`tamaño = 4`

`tamaño() = 4`

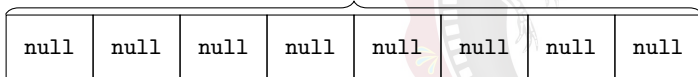
Método `estaVacia()`

Definición

Indica si la lista **esta vacía**.

1: **return** `tamano = 0`

`elementos.length = 8`



0

1

2

3

4

5

6

7

`tamano = 0`

`estaVacia() = true`

Método limpiar()

Definición

Vacía el contenido de la lista

- 1: **for** $i \leftarrow 0$ **hasta** $\text{tamano} - 1$ **do**
- 2: $\text{elementos}[i] \leftarrow \text{null}$
- 3: **end for**
- 4: $\text{tamano} \leftarrow 0$

limpiar()

10	20	30	40	null	null	null	null
----	----	----	----	------	------	------	------

↑
 $i = 0$

null	20	30	40	null	null	null	null
------	----	----	----	------	------	------	------

↑
 $i = 1$

Método limpiar()

null	null	30	40	null	null	null	null
------	------	----	----	------	------	------	------

↑
 $i = 2$

null	null	null	40	null	null	null	null
------	------	------	----	------	------	------	------

↑
 $i = 3$

null	null	null	null	null	null	null	null
------	------	------	------	------	------	------	------

tamaño = 0

Fin de la presentación

¿Preguntas o comentarios?

